

Método de Planejamento — MWRPG

Adaptado do método estabelecido no PushProcessos

(PushProcessos/docs/METODO-PLANEJAMENTO.md), que o Tiago definiu como "framework" padrão de produção em todos os projetos que conduz com Claude Code. Este documento é a versão aplicada ao domínio de jogo do MWRPG — mesma espinha dorsal, adaptada onde o domínio pede.

O que isso é, com honestidade

Os "agentes" consultados abaixo são personas do `.claude/agents/` deste projeto, cada uma com um ângulo de especialidade (design de sistema de jogo, narrativa, multiplayer em tempo real, IA do mestre/NPCs, backend, frontend, infra, qualidade, testes). **Não são pareceres independentes** — é o mesmo modelo (Claude) sendo instruído a examinar o mesmo problema por lentes diferentes, deliberadamente, pra forçar que nenhum ângulo importante fique de fora antes de decidir. A "votação" não é uma democracia entre partes neutras: é um jeito estruturado de expor trade-offs e registrar onde as lentes concordam e onde discordam. O valor está no exame multiangular, não na contagem de votos em si — o resultado nunca deve ser apresentado como "a maioria decidiu" sem também mostrar as objeções da minoria que tiverem peso técnico real.

Quando usar

Funcionalidade nova que: (a) introduz entidade de dado nova (sala, personagem, campanha, sessão multiplayer), ou (b) muda comportamento visível do jogador de forma não-trivial, ou (c) tem risco técnico real de não caber na fundação atual (zero-build, custo do LLM por sessão, tier gratuito de hosting/realtime). Não usar para: correção de bug, ajuste de copy narrativa, refino visual, tarefa com escopo já decidido numa conversa anterior.

Passo a passo

- 1 Pesquisar antes de planejar.** Preço de API de LLM, limites de tier free de hosting/realtime, e qualquer afirmação técnica citável — nunca por memória. "Modelo errado é pior que nenhum" vale tanto para preço de infra quanto para regra de domínio jurídico no PushProcessos.
- 2 Explorar a fundação atual** — ler o protótipo já construído (`src/`, `CLAUDE.md`, `README.md`, `Relatorio_Pesquisa_RPG.md`) antes de desenhar qualquer coisa nova, pra identificar o que já funciona (sistema D6 das Três Letras, contrato JSON do mestre, design system "Manuscrito Vivo") e não deve ser redesenhado sem motivo.
- 3 Rascunhar um baseline próprio**, nesta ordem: Planejamento geral → Mapa de Implementação (fases) → Mapa de Estruturas (onde cada peça nova mora) → Mapa de Entidades (sala, personagem, campanha, membro — campo a campo). Esse baseline é só munição pro debate, não uma proposta fechada.
- 4 Consulta individual aos agentes envolvidos** — um agente por vez, pedindo um plano estruturado na hierarquia: **Objetivos** → **Planejamento** → **Processos** → **Tarefas** → **Envolvidos e Atividades**, com liberdade explícita pra criticar o baseline e divergir dele.

- 5 **Assembleia conjunta (sintetizada por quem conduz, não uma reunião real)** — cruzar as respostas individuais: o que convergiu vira parte fixa de qualquer plano final; onde há divergência real, isso vira um eixo de variação.
- 6 **5 finalistas** — compor 5 planos completos e coerentes, cada um uma combinação diferente e sensata dos eixos de variação encontrados. Não é combinatória cega — cada finalista precisa fazer sentido como estratégia própria (ex.: "MVP mínimo", "equilíbrio", "aposta em X").
- 7 **Votação** — voltar a cada agente com os 5 finalistas resumidos, pedindo o voto e a justificativa do próprio ângulo, e se algum finalista tem um problema que o desclassifica (não só "é pior") do ponto de vista daquele agente.
- 8 **Apresentar o vencedor** — tally dos votos + as objeções de peso da minoria que não podem ser ignoradas mesmo perdendo a votação. O plano final é o vencedor **com as mitigações da minoria incorporadas** quando forem baratas e tecnicamente sólidas.
- 9 **Esperar aprovação explícita do Tiago antes de implementar.** Nenhuma linha de código do recurso em si até esse sinal — só a própria implementação deste método (este documento e a referência no CLAUDE.md) não depende de aprovação, porque é processo, não produto.

Regra permanente: o ciclo depois de resolver algo localmente

Mesma regra do PushProcessos, vale igual aqui: resolver um problema — feature, bug, correção de copy — **localmente não é o fim do trabalho**. Depois de qualquer solução validada local, o ciclo obrigatório é:

- 1 **Commit** da mudança.
- 2 **Push** para main — dispara o deploy automático (Vercel, quando configurado).
- 3 **Testar de verdade na URL de produção** — nunca dar por confirmado só porque funcionou local.
- 4 **Atualizar a documentação afetada** (README.md, CLAUDE.md, e os docs de docs/ que citarem o mecanismo mudado).

Regra específica do domínio de jogo: teste manual real no navegador

Um jogo narrativo com estado (sala, personagem, campanha) tem a mesma categoria de risco que o PushProcessos identificou em telas que ficam abertas depois de salvar: uma sessão de jogo se estende por múltiplos turnos encadeados, então testes "isolados" (sempre partindo de estado fresco) não pegam bugs de acúmulo de estado ao longo de uma sessão real. **Regra:** qualquer feature de sessão/sala precisa de teste manual real no navegador que jogue vários turnos seguidos, não só "abre e faz uma ação".

Registro do primeiro uso

Primeira aplicação: assembleia sobre a arquitetura da plataforma multiplayer (salas, mestre IA em produção, NPCs, persistência de campanha), 31/08/2026. Ver docs/ASSEMBLEIA-01-PLATAFORMA-MULTIPLAYER.md.

Correção de rumo (31/08/2026)

Entre a Assembleia 02 e a Assembleia 03, várias decisões de porte relevante (login, senha pós-login, limite de demo, contador por campanha, sistema de mapa) foram implementadas direto, sem passar pelo método — cobrança justa do Tiago. Auditoria completa em docs/RETROSPECTIVA-01-DESVIO-DE-METODO.md; o método volta a valer sem exceção a partir da docs/ASSEMBLEIA-03-PERSONAGEM-E-ORCAMENTO.md.